Informe Técnico: Capa Web del Catálogo con Servlets y JSP

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Esta evidencia codifica el módulo enfocado a web del proyecto **MONK GRAIN**: formularios HTML procesados por **servlets**, con los métodos **GET** y **POST**, y elementos de **JSP** para la presentación.

No parte de cero. La capa web **reutiliza como dependencia** el módulo `catalogo-pergamino` de la evidencia AA2-EV01: los servlets no repiten el acceso a datos ni las reglas de negocio, los invocan. Esa reutilización es precisamente lo que hace que la división en capas valga la pena, y queda demostrada aquí: la misma regla sirve a dos interfaces —consola y web— sin duplicarse.

2. Objetivo

Codificar el módulo enfocado a web con servlets: formularios HTML procesados por servlets, con métodos GET y POST, y elementos de JSP, cumpliendo los estándares de codificación del proyecto y versionando el trabajo con Git.

3. Arquitectura: una capa más sobre lo ya construido

Componente	Responsabilidad
`CatalogServlet`	Catálogo público. Solo GET
`LotServlet`	Gestión de lotes. GET para mostrar, POST para crear y eliminar
JSP	Presentación. No contienen lógica de negocio
`catalogo-pergamino`	Modelo, DAO y reglas, reutilizados de AA2-EV01

> **Los servlets no consultan la base de datos.** Recogen los parámetros, los convierten, y delegan. Si mañana cambia la regla del filtro, cambia en un solo sitio y las dos interfaces la heredan.

4. Los dos métodos HTTP

4.1 GET — consultar

El filtro del catálogo va por GET para que el criterio quede en la dirección y la página pueda compartirse o recargarse:

```
@WebServlet(name = "CatalogServlet", urlPatterns = {"/catalogo"})
public class CatalogServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String parametro = request.getParameter("factorMaximo");

        try {
            BigDecimal factorMaximo = parsearFactor(parametro);
            List<PergaminoLot> lotes = service.browseCatalog(factorMaximo);

            request.setAttribute("lotes", lotes);
            request.setAttribute("total", lotes.size());

            request.getRequestDispatcher("/WEB-INF/jsp/catalogo.jsp")
                   .forward(request, response);

        } catch (ValidationException e) {
            // Un filtro mal escrito es culpa de quien lo escribe: 400, no 500.
            reenviarAError(request, response, e.getMessage(),
                    HttpServletResponse.SC_BAD_REQUEST);
        }
    }
}
```

4.2 POST — modificar

Tras un POST que modifica datos se responde con una **redirección**, no con el contenido. Es el patrón POST-Redirect-GET, y no es una formalidad: sin él, recargar la página reenvía el formulario y el lote se publica dos veces.

```
private void crear(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

    PergaminoLot lote = new PergaminoLot();
    lote.setProducerId(parsearEntero(request.getParameter("productor"), "vendedor"));
    lote.setLotCode(textoObligatorio(request.getParameter("codigo"), "código del lote"));
    lote.setYieldFactor(parsearDecimalOpcional(
            request.getParameter("factor"), "factor de rendimiento"));
    ...
    int id = service.registerLot(lote);

    // POST-Redirect-GET: recargar no debe publicar el lote dos veces.
    response.sendRedirect(request.getContextPath()
            + "/lotes?mensaje=" + codificar("Lote " + lote.getLotCode()
            + " publicado con el identificador " + id + "."));
}
```

> **Eliminar va por POST, nunca por un enlace.** Una acción que borra no debe poder dispararse con una visita: un rastreador que siga los enlaces de la página vaciaría el catálogo.

4.3 La codificación de caracteres

```
request.setCharacterEncoding("UTF-8");
```

Sin esa línea al principio de `doPost`, los acentos del formulario llegan rotos y «Nariño» se guarda como «NariÃ±o». El descriptor de despliegue lo refuerza con `<request-character-encoding>`.

5. Conversión y validación de la entrada

Toda la entrada de un formulario llega como **texto**. Convertirla con un mensaje que nombra el campo evita que el usuario reciba un `NumberFormatException`, que no le dice qué corregir:

```
private BigDecimal parsearDecimalOpcional(String valor, String campo) {
    if (valor == null || valor.isBlank()) {
        return null;
    }
    try {
        // Se admite la coma decimal: es como se escribe en Colombia.
        return new BigDecimal(valor.trim().replace(',', '.'));
    } catch (NumberFormatException e) {
        throw new ValidationException(
                "El " + campo + " debe ser un número. Recibimos: «" + valor + "».");
    }
}
```

El formulario también valida en el navegador (`required`, `min`, `max`, `step`), pero eso es una comodidad, no una defensa: la comprobación del navegador se salta con una petición directa, como demuestran las pruebas de la sección 8.

6. Elementos de JSP utilizados

Elemento	Uso	Dónde
Directiva `page`	Tipo de contenido y codificación	Todas
Directiva `taglib`	Importa JSTL core y fmt	`catalogo.jsp`, `lotes.jsp`
Acción `jsp:include`	Cabecera y pie comunes	Todas
Lenguaje de expresiones `${...}`	Lee atributos de la petición	Todas
`c:forEach`	Recorre la lista de lotes	`catalogo.jsp`
`c:if` y `c:choose`	Mostrar u ocultar según el dato	`catalogo.jsp`, `lote-detalle.jsp`
`fmt:formatNumber`	Moneda y decimales	`catalogo.jsp`
`isErrorPage="true"`	Acceso al objeto `exception`	`error.jsp`

Un fragmento representativo, del catálogo:

```
<c:forEach var="lote" items="${lotes}">
  <tr>
    <td><code>${lote.lotCode}</code></td>
    <td class="num">
      <c:choose>
        <c:when test="${empty lote.yieldFactor}">—</c:when>
        <c:otherwise>
          <fmt:formatNumber value="${lote.yieldFactor}" minFractionDigits="2"/>
        </c:otherwise>
      </c:choose>
    </td>
    <td><span class="etiqueta">${lote.yieldFactorLabel}</span></td>
    <td class="num">${lote.availableAsCargas}</td>
  </tr>
</c:forEach>
```

> **Las JSP viven bajo `WEB-INF/`**, que el contenedor no sirve directamente. Solo se alcanzan por el `forward` de un servlet, de modo que nadie puede saltarse la lógica escribiendo la ruta de la página.

7. Un defecto encontrado al desplegar

El módulo funcionaba por consola y, al desplegarlo en Tomcat, el catálogo devolvía **500**:

```
SEVERE [http-nio-8080-exec-1] Servlet.service() for servlet [CatalogServlet]
  co.monkgrain.catalog.util.DataAccessException: No fue posible conectar con la
  base de datos en jdbc:postgresql://localhost:5432/monkgrain_java
  Caused by: java.sql.SQLException: No suitable driver found
```

El controlador **sí estaba** en el WAR (`WEB-INF/lib/postgresql-42.7.4.jar`). La causa es más sutil: desde JDBC 4.0 el controlador se registra solo, pero `DriverManager` hace esa búsqueda **una sola vez**, con el cargador de clases activo en ese momento. Si Tomcat tocó `DriverManager` antes que la aplicación —lo habitual—, la búsqueda ya ocurrió sin ver el controlador que vive en `WEB-INF/lib`.

La corrección, en el módulo de la evidencia anterior:

```
static {
    try {
        Class.forName(CONTROLADOR);
    } catch (ClassNotFoundException e) {
        throw new DataAccessException(
                "No se encontró el controlador JDBC " + CONTROLADOR
                        + ". Verifica que el jar de PostgreSQL esté en el classpath.", e);
    }
    ...
}
```

`Class.forName` fuerza el registro desde el cargador de la aplicación web. En consola es inofensivo, de modo que el módulo sigue funcionando en los dos entornos: se comprobó volviendo a ejecutar sus 13 pruebas y la demostración por consola.

8. Pruebas en ejecución

Desplegado en **Apache Tomcat 10.1.61** y ejercido por HTTP real, no por inspección del código.

8.1 Respuestas correctas

```
GET  /catalogo                    → 200
GET  /lotes                       → 200
GET  /lotes?accion=nuevo          → 200

POST /lotes (registrar vendedor)  → 302 → /lotes?accion=nuevo&mensaje=Vendedor+...
POST /lotes (publicar MIR-2026-001) → 302
POST /lotes (publicar MIR-2026-002) → 302

GET  /catalogo
     MIR-2026-001 · Excepcional · 1250 kg (10 cargas)
     MIR-2026-002 · Estándar    · 375 kg (3 cargas)

GET  /catalogo?factorMaximo=92    → 1 lote: MIR-2026-001
     (el filtro acota el TECHO: menos es mejor)

GET  /lotes?accion=detalle&id=7  → Excepcional · en el rango seguro
POST /lotes (eliminar)            → 302
```

8.2 Validaciones, saltándose el navegador

Estas peticiones se enviaron **directamente por HTTP**, sin pasar por el formulario, que es justo lo que demuestra que la validación del servidor no es decorativa:

Entrada	Código	Mensaje devuelto
Factor 145	`400`	El factor de rendimiento debe estar entre 80 y 130. Un valor fuera de ese rango es un error de digitación, no un café excepcional.
Código repetido	`400`	Ya tienes un lote con el código MIR-2026-001.
Precio `abc`	`400`	El precio por kilogramo debe ser un número. Recibimos: «abc».
Fecha `15-09-2026`	`400`	La fecha de cosecha debe tener el formato AAAA-MM-DD. Recibimos: «15-09-2026».
Humedad 150 %	`400`	La humedad debe estar entre 0 % y 100 %.
Vendedor inexistente	`400`	No existe el vendedor 1.
Filtro `abc` (GET)	`400`	El factor de rendimiento debe ser un número. Recibimos: «abc».

El código `400` y no `500` es intencional: un dato mal escrito es culpa de quien lo escribe y se corrige cambiándolo; un `500` diría que el fallo es del servidor y que no hay nada que el usuario pueda hacer.

> **Lo que puedes hacer tú**
>
> **Ver el módulo en el navegador.** Tomcat quedó corriendo con la aplicación desplegada: **http://localhost:8080/catalogo-pergamino/catalogo**. Si lo apagaste, se levanta con `bin/startup.sh` desde la carpeta de Tomcat. Para las capturas de la evidencia AA3-EV02 conviene recorrer el catálogo, el formulario y un error de validación.

9. Estructura del módulo

```
catalogo-web/
├── pom.xml                      empaquetado WAR, depende de catalogo-pergamino
└── src/main/
    ├── java/co/monkgrain/catalog/web/
    │   ├── CatalogServlet.java     GET  /catalogo
    │   └── LotServlet.java         GET y POST  /lotes
    └── webapp/
        ├── index.jsp               redirige al catálogo
        ├── estilo.css
        └── WEB-INF/
            ├── web.xml             bienvenida, codificación, páginas de error
            └── jsp/
                ├── _cabecera.jsp   incluida por todas
                ├── _pie.jsp
                ├── catalogo.jsp    catálogo público con filtro
                ├── lotes.jsp       bandeja del vendedor
                ├── lote-formulario.jsp
                ├── lote-detalle.jsp
                └── error.jsp
```

10. Conclusión

El módulo web implementa formularios HTML procesados por servlets con GET y POST, y emplea los elementos de JSP —directivas, acciones, lenguaje de expresiones y JSTL— para la presentación, sin lógica de negocio en las páginas.

Lo que mejor demuestra la arquitectura es lo que **no** hubo que escribir: ni una consulta SQL ni una regla de validación. Todo eso vino del módulo de la evidencia anterior, y la única corrección que hizo falta —la carga explícita del controlador JDBC— se aplicó allí y benefició a las dos interfaces.